

中山大學

SUN YAT-SEN UNIVERSITY

实验 0x05 实验报告

fork 的实现、并发与锁机制

姓　　名：　何亮

学　　号：　24325005

教学班号：　202520804（陈鹏飞老师班级）

专　　业：　计算机科学与技术

院　　系：　计算机学院

2025 学年第二学期

1. fork 的实现

1.1. 系统调用

首先, 我们设置系统调用号为 `58`, 将用户态库中给出的系统调用对接到内核对应的函数当中，部分代码如下

```
#[inline(always)]
pub fn sys_fork() -> u16 {
    syscall!(Syscall::Fork) as u16
}

Syscall::Fork => {
    crate::proc::fork(context);
}
```

1.2. 进程管理

首先，我们实现最外层的 `fork`

```
pub fn fork(context: &mut ProcessContext) {
    x86_64::instructions::interrupts::without_interrupts(|| {
        let manager = get_process_manager();
        // FIXED: save_current as parent
        manager.save_current(context);
        // FIXED: fork to get child
        manager.fork();
        // FIXED: push to child & parent to ready queue
        // FIXED: switch to next process
        manager.switch_next(context);
    })
}
```

具体而言，就是保存当前上下文，然后调用 `ProcessManager::fork` 委托给下一层处理，然后再切换到下一进程的上下文，这里的将子进程放入就绪队列的操作在下层 `fork` 实现。

然后，实现 `ProcessManager::fork`

```rust
pub fn fork(&self) {
    // FIXED: get current process
    let current = self.current();
    // FIXED: fork to get child
    let child = current.fork();
    let child_pid = child.pid();
    // FIXED: add child to process list
    self.add_proc(child_pid, child.clone());
    self.push_ready(child_pid);
    // FOR DBG: maybe print the process ready queue?
    log::trace!(
        "Process {:?} successfully forked child {:?}",
        current.pid(),
        child_pid
    );
}
```

这里是将当前的 `fork` 任务委托到下一层进程自己的 `Process::fork` 执行，然后本层要做的就是把子进程放入就绪队列并加入进程管理表中。

接着实现 `Process::fork`

```rust
pub fn fork(self: &Arc<Self>) -> Arc<Self> {
    // FIXED: lock inner as write
    let mut inner = self.write();
    // FIXED: inner fork with parent weak ref
    let child_inner = inner.fork(Arc::downgrade(self));

    // FOR DBG: maybe print the child process info
    //          e.g. parent, name, pid, etc.
    log::trace!(
        "Process {}#{} forked a child process with pid: {}",
        inner.name(),
        self.pid,
        child_inner.parent().unwrap().pid()
    );
    // FIXED: make the arc of child
    let child = Arc::new(Self {
        pid: ProcessId::new(),
        inner: spin::RwLock::new(child_inner),
    });
    // FIXED: add child to current process's children list
    inner.children.push(child.clone());
    // FIXED: set fork ret value for parent with `context.set_rax`
    inner.context.set_rax(child.pid().0 as usize);
    // FIXED: mark the child as ready & return it
    child
}
```

这一层需要做的是，委托到 `ProcessInner::fork`，然后把当前获得的子进程的 `ProcessInner` 和其 `pid` 打包，并放入当前进程的子进程表中，并设置父进程的 `rax` 返回值寄存器为子进程的 `pid`。

然后实现的是 `ProcessInner::fork`

```
pub fn fork(&mut self, parent: Weak<Process>) -> ProcessInner {
    // FIXED: fork the process virtual memory struct
    // FIXED: calculate the real stack offset
    let stack_offset_count = self.children.len() as u64 + 1;
    let child_vm = self.vm().fork(stack_offset_count);
    // FIXED: clone the process data struct
    let mut child_context = self.context.clone();

    // FIXED: update `rsp` in interrupt stack frame
    // FIXED: set the return value 0 for child with `context.set_rax`

    // let stack_offset = stack_offset_count * STACK_MAX_SIZE;
    let parent_stack_base = self.vm().stack.range.start.start_address().as_u64();
    let child_stack_base = child_vm.stack.range.start.start_address().as_u64();
    let stack_offset = parent_stack_base - child_stack_base;

    child_context.as_mut().stack_frame.stack_pointer -= stack_offset;
    child_context.as_mut().regs.rbp -= stack_offset as usize;
    child_context.set_rax(0);
    // FIXED: construct the child process inner
    let child_data = self.proc_data.clone();
    // NOTE: return inner because there's no pid record in inner
    ProcessInner {
        name: self.name.clone(),
        parent: Some(parent),
        children: Vec::new(),
        ticks_passed: 0,
        status: ProgramStatus::Ready,
        context: child_context,
        exit_code: None,
        proc_data: child_data,
        proc_vm: Some(child_vm),
    }
}
```

这一层需要做的是，委托到内存管理器，建立对应的栈，然后本层做的是计算出对应的栈指针地址，这里需要 `+1` 是因为上层 `fork` 是先调用了当前的 `fork` 之后再把子进程插入子进程 `Vec` 中，此后需要修改栈指针，设置子进程的 `rax` 返回值寄存器的值为 `0`，然后构造出子进程的 PCB 并返回。

在实现的时候，本来使用 `stack_offset_count * STACK_MAX_SIZE` 计算偏移的，但是在测试的时候出现了地址的冲突，原因在于 `map_range` 内部会调整冲突的偏移，因此需要用父子进程的栈起始地址之差进行计算。

最后要实现的就是栈的建立和复制了，如下

```
pub fn fork(
    &self,
    mapper: MapperRef,
    alloc: FrameAllocatorRef,
    stack_offset_count: u64,
) -> Self {
    // FIXED: alloc & map new stack for child (see instructions)
    let cur_stack_base = self.range.start.start_address().as_u64();
    let mut new_stack_base = cur_stack_base - stack_offset_count *
STACK_MAX_SIZE;

    // log::info!("1. current: {}, new: {}", cur_stack_base, new_stack_base);
    // log::info!("usage: {} range: {}", self.usage, cur_stack_base);

    while elf::map_range(new_stack_base, self.usage, mapper, alloc).is_err() {
        trace!("Map thread stack to {:#x} failed.", new_stack_base);
        new_stack_base -= STACK_MAX_SIZE; // stack grow down
    }
    // log::info!("2. current: {}, new: {}", cur_stack_base, new_stack_base);
    // FIXED: copy the *entire stack* from parent to child
    self.clone_range(cur_stack_base, new_stack_base, self.usage);
    // FIXED: return the new stack
    Self {
        range: Page::range(
            Page::containing_address(VirtAddr::new(new_stack_base)),
            Page::containing_address(VirtAddr::new(new_stack_base)) + self.usage,
        ),
        usage: self.usage,
    }
}
```

具体而言，就是先计算出子进程的栈的栈底，然后建立页表映射，最后复制父进程的数据，把栈的 `range` 和 `usage` 信息打包后返回。

然后尝试运行 `fork_test` 进行测试，

```
[>] fork_test
I am the child process
I am the parent process
  PID | PPID | Process Name |  Ticks   | Status | Memory Usage
 #  3 | #  2 | fork_test    |        4 | Running | 28672 B (7 Pages)
 #  2 | #  1 | sh           |    13201 | Ready | 28672 B (7 Pages)
 #  4 | #  3 | fork_test    |        3 | Ready | 28672 B (7 Pages)
 #  1 | #  0 | kernel       |    13216 | Ready | 2097152 B (512 Pages)
Queue  : [4, 2, 1]
CPUs   : [0: 3]
Waiting for child to exit...
child read value of M: 0xdeadbeef
child changed the value of M: 0x2333
Child exited with status 64
parent read value of M: 0x2333
Process [3] exited with code: 1056
```

可以看到，`fork_test` 正常运行并退出，可知 `fork` 的实现大致无误。

2. 进程的阻塞与唤醒

这个部分的实现比较简单，主要就是把 `wait_pid` 从轮询的方式改为阻塞，每次将一个需要等待的进程放入被等待进程的等待队列中然后阻塞（设置为 `blocked` 状态）。当那个被等待的进程被 `kill` 了之后，把所有等待它的进程唤醒，将状态改为 `ready` 并放入就绪队列。

实现了基于阻塞的 `wait_pid` 后进行测试，可以看到原本的 `sh` 被阻塞，当当前的 `sh` 退出后，又调度回到 `running` 态，可知这部分功能实现大致无误。

```
Welcome to YatSenOS Shell!
Type 'help' for instructions.
[>] sh
Welcome to YatSenOS Shell!
Type 'help' for instructions.
[>] stat
  PID | PPID | Process Name |  Ticks   | Status | Memory Usage
 #  3 | #  2 | sh           |   14418 | Running | 28672 B (7 Pages)
 #  1 | #  0 | kernel       |   22969 | Ready | 2097152 B (512 Pages)
 #  2 | #  1 | sh           |    8542 | Blocked | 28672 B (7 Pages)
Queue  : [1]
CPUs   : [0: 3]
[>] exit
Goodbye!
Process [3] exited with code: 0
[>] stat
  PID | PPID | Process Name |  Ticks   | Status | Memory Usage
 #  1 | #  0 | kernel       |  225334 | Ready | 2097152 B (512 Pages)
 #  2 | #  1 | sh           |   17514 | Running | 28672 B (7 Pages)
Queue  : [1]
CPUs   : [0: 2]
[>] exit
Goodbye!
[INFO] ysos_kernel - YatSenOS shutting down.
```

3. 并发与锁机制

4. 思考题

4.1. 在 Lab 2 中设计输入缓冲区时，如果不使用无锁队列实现，而选择使用 `Mutex` 对一个同步队列进行保护，在编写相关函数时需要注意什么问题？考虑在进行 `pop` 操作过程中遇到串口输入中断的情形，尝试描述遇到问题的场景，并提出解决方案。

需要注意会不会出现死锁，比如当 `pop` 操作进行时，此时内核取得了缓冲区得读写锁，此时遇到串口输出中断，中断处理程序抢占 `CPU` 自旋等待读写锁写入缓冲区，但此时的锁在内核手上无法释放，就造成了死锁。解决的方法时，在内核进行 `pop` 操作时，需要先关中断，防止其他中断自旋等待该锁造成死锁。

4.2. 在进行 `fork` 的复制内存的过程中，系统的当前页表、进程页表、子进程页表、内核页表等之间的关系是怎样的? 在进行内存复制时，需要注意哪些问题?

在一般的 OS 的 `fork` 复制内存过程中，系统的当前页表指向父进程的页表，子进程页表采用 Cow 写时复制机制，内核页表共享。本实验中的简化实现是子进程和父进程共享一张页表，然后子进程设置自己独立的栈和寄存器。需要注意的是在实验文档中设计的这种栈布局在复杂情况下可能会造成栈复用，导致 `map_range` 失败，从而会带来偏移量的调整，因此在计算栈的偏移的时候需要以最终映射成功的内存地址之差来计算偏移，否则会造成内存复制时的冲突。

4.3. 为什么在实验的实现中，`fork` 系统调用必须在任何 `Rust` 内存分配（堆内存分配）之前进行？如果在堆内存分配之后进行 `fork`，会有什么问题?

因为我们的实现中，没有实现 Cow 写时复制机制，父子进程的堆是共享的，`fork` 只会给子进程建立自己的栈，因此 `fork` 系统调用需要在任何的堆内存分配之前进行，来保证子进程退出后 drop 了的堆变量是子进程自己的，而不是和父进程共享的。如果先进行了堆内存分配后 `fork` 则可能会带来 double-free 或者 use-after-free 的问题。

4.4. 进行原子操作时候的 `Ordering` 参数是什么？此处 Rust 声明的内容与 C++20 规范 中的一致，尝试搜索并简单了解相关内容，简单介绍该枚举的每个值对应于什么含义。

`Ordering` 参数对应的当前进程的指令顺序的排序严格程度。`Relaxed` 对应的是只保证当前操作是原子的，但是并不保证在其他进程的观测中中这些原子操作的顺序不变；`Acquire` 一般与 `Load` 结合使用，对应的是保证在其他线程的观测中，可以确定在这个 `Load` 操作之后的读写操作不会指令重排到其之前；`Release` 一般与 `Store` 结合使用，对应的是保证在其他线程的观测中，可以确定在这个 `Store` 指令之前的读写操作不会被重排到其之后，可以结合如下代码理解（参考自知乎用户 newbmiao 的回答）

```rust
let a = thread::spawn(|| {
    unsafe { DATA = 42 };
    FLAG.store(true, Ordering::Release);
    // 不允许有读写重排到 store 之后
    // 即后面的 FLAG 变为 true 之前一定已经写入了 42
});
let b = thread::spawn(|| {
    // 不允许有读写重排到 load 之前
    // 即后面的 DATA 读取到的一定是 42
    while !FLAG.load(Ordering::Acquire) {}
    assert_eq!(unsafe { DATA }, 42);
});

a.join().unwrap();
b.join().unwrap();
```

而 `AcqRel` 是上述两者的结合，一般用在 `fetch_add` 等原子操作上；最严格的 `SeqCst` 是在保证上述重排限制的前提下，另外保证所有线程都可以按相同的顺序看到所有顺序一致的操作（即前者可以保证的是单一原子变量各个原子操作之间的顺序，后者可以保证全局所有原子变量的操作之间的顺序）。如下方的代码，如果不使用 `SeqCst`，可能会导致两个 `Thread...` 都输出的情况，即使使用 `Acquire + Release` 也没用，因为这个只保证线程内部的 `Store` 一定在 `Load` 之前，但不能保证两个线程判断时看到的 `X, Y` 的修改顺序一致，除此以外，还要求所有 `SeqCst` 操作在所有线程看来有一个相同的全序（即不可能出现两个都能打印出来的情况）

```rust
let t1 = thread::spawn(|| {
    X.store(true, Ordering::SeqCst);
    if !Y.load(Ordering::SeqCst) {
        println!("Thread 1: X was set before Y was observed false.");
    }
});

let t2 = thread::spawn(|| {
    Y.store(true, Ordering::SeqCst);
    if !X.load(Ordering::SeqCst) {
        println!("Thread 2: Y was set before X was observed false.");
    }
});

t1.join().unwrap();
t2.join().unwrap();
```

4.5. 在实现 `SpinLock` 的时候，为什么需要实现 `Sync` trait？类似的 `Send` trait 又是什么含义？

`Sync` 的含义是如果一个类型 `T` 实现了 `Sync` trait, 就表示 `&T` 不可变引用可以在多个线程之间安全地共享, 即多个线程可以同时通过不可变引用来访问同一个值而不会引发数据竞争, 如果不实现这个 trait 会通过不了编译。而 `Send` 指的是如果一个类型 `T` 实现了 `Send` trait, 就表示它的所有权可以在线程之间安全地转移，即可以把一个 `T` 类型的值从一个线程 `move` 到另一个线程。即前者是可以共享使用这个锁，而后者是可以传递这个锁的所有权。

4.6. `core::hint::spin_loop` 使用的 `pause` 指令和 Lab 4 中的 `x86_64::instructions::hlt` 指令有什么区别？这里为什么不能使用 `hlt` 指令?

`hlt` 指令指的是 `Halt`，这是一个特权指令，会直接让 CPU 休眠直到一个外部中断来临；而 `pause` 指令是一个非特权指令，会让 CPU 会优化执行，提供较少的资源给当前线程，减少自旋等待对 CPU 的消耗。这里由于 `counter` 是一个用户态程序，不能使用 `hlt` 这种特权指令；其次 `hlt` 需要外部中断才会唤醒，如果在关中断等特殊的时候使用可能会造成永久睡眠。